BTS CIEL · PROGRAMMATION C++ · TRAVAUX PRATIQUES

TP07 — Gestion d'un parc de capteurs

Prérequis : chapitres 1 à 7

Contexte professionnel

Un bâtiment tertiaire est équipé de capteurs de température, d'humidité et de pression, supervisés depuis le poste du technicien. Chaque capteur possède une plage de fonctionnement normal (seuils bas et haut) ; un capteur **actif** dont la mesure sort de sa plage est **en alarme**. Un capteur peut être désactivé pendant une maintenance : il n'est alors plus surveillé.

Vous développez le module console de gestion du parc : consultation, recherche, mise à jour des mesures, activation et liste des alarmes.

Objectifs

- Modéliser un équipement avec une `struct` et un parc avec un `std::vector`.
- Choisir et justifier le mode de passage de chaque paramètre : valeur, référence, référence constante ou pointeur.
- Exprimer « trouvé ou absent » avec un pointeur qui peut valoir `nullptr`.
- Organiser un programme interactif en fonctions.

Compétences mobilisées

Compétence	Mise en œuvre dans ce TP
C04 — Analyser un système informatique	Identifier les données d'un capteur et les règles d'alarme
C05 — Concevoir un système informatique	Concevoir les fonctions et leurs signatures, justifier les passages de paramètres
C06 — Valider un système informatique	Dérouler un scénario de test et vérifier chaque état du parc
C08 — Coder	Écrire la structure, les fonctions et le menu

Cahier des charges

Données d'un capteur

```
struct Capteur {
    std::string identifiant{};
    std::string type{};        // "temperature", "humidite" ou "pression"
    double valeur{};
    double seuilMin{};
    double seuilMax{};
    bool actif{};
};
```

Parc initial (fourni dans le fichier de départ)

Identifiant	Type	Valeur	Seuils	État
T1	temperature	21.5	15 à 27	actif
T2	temperature	29	15 à 27	actif
H1	humidite	45	30 à 60	actif
H2	humidite	72	30 à 60	inactif
P1	pression	1013	950 à 1050	actif

Menu (répété jusqu'au choix 0)

Choix	Traitement
1. Afficher le parc	Une ligne par capteur : identifiant, type, valeur, seuils, état ; mention `ALARME` si le capteur est en alarme
2. Rechercher un capteur	Saisie d'un identifiant ; affiche la ligne du capteur, ou `Capteur XX introuvable`
3. Mettre à jour une mesure	Saisie d'un identifiant puis (s'il existe) de la nouvelle valeur, saisie robuste ; affiche `T2 : 29 -> 24`
4. Activer / désactiver	Saisie d'un identifiant ; inverse l'état ; affiche `H2 active` ou `H2 desactive`
5. Alarmes	Liste les capteurs en alarme, puis `N alarme(s)`
6. Moyenne par type	Saisie d'un type ; moyenne des capteurs **actifs** de ce type, ou `Aucun capteur actif de type X`
0. Quitter	—

Un capteur est **en alarme** s'il est actif et si sa valeur est strictement inférieure à son seuil bas ou strictement supérieure à son seuil haut.

Fonctions à écrire

Fonction	Rôle
`estEnAlarme`	Indique si un capteur est en alarme
`afficherCapteur`	Affiche la ligne d'un capteur
`afficherParc`	Affiche tout le parc
`chercherCapteur`	Renvoie l'adresse du capteur d'identifiant donné, ou `nullptr`
`basculerActivation`	Inverse l'état d'un capteur
`moyenneParType`	Calcule la moyenne des capteurs actifs d'un type ; renvoie `false` s'il n'y en a aucun
`lireReel`	Saisie robuste d'un nombre décimal ; renvoie `false` si l'entrée est fermée

Travail demandé

Partie 1 — Conception

1. Écrivez le prototype de chaque fonction du tableau et justifiez **chaque** paramètre (valeur, référence, référence constante ou pointeur) et le type de retour.
2. Pour le scénario de test ci-dessous, prévoyez l'affichage de chaque choix.

Partie 2 — Consultation

3. À partir du fichier de départ `parc_capteurs.cpp`, écrivez `estEnAlarme`, `afficherCapteur`, `afficherParc`, `chercherCapteur`, puis les choix 1, 2 et 5 du menu.

Partie 3 — Modifications

4. Écrivez `lireReel`, `basculerActivation`, `moyenneParType`, puis les choix 3, 4 et 6.

Partie 4 — Validation

5. Déroulez le scénario de test et les cas particuliers.

Contraintes techniques

- Compilation avec `g++ -std=c++17 -Wall -Wextra`, sans erreur ni avertissement.
- Aucune copie inutile d'un `Capteur` ou du parc ; aucun pointeur là où une référence suffit.
- Tout pointeur renvoyé par `chercherCapteur` est testé avant d'être utilisé, et utilisé avec `->`.
- Aucun capteur n'est ajouté au parc pendant l'exécution (un `push_back` pourrait invalider un pointeur obtenu par `chercherCapteur`).
- Les identifiants et les types saisis sont des mots sans espace.

Tests à effectuer

Scénario principal

Étape	Choix et saisies	Résultat attendu
1	`5`	`T2` en alarme ; `1 alarme(s)`
2	`4` `H2`	`H2 active`
3	`5`	`T2` et `H2` en alarme ; `2 alarme(s)`
4	`3` `T2` `24`	`T2 : 29 -> 24`
5	`5`	`H2` en alarme ; `1 alarme(s)`
6	`6` `temperature`	`Moyenne temperature : 22.75`
7	`2` `X9`	`Capteur X9 introuvable`
8	`0`	Fin du programme, code de retour 0

Cas particuliers

N°	Situation	Résultat attendu
2	Choix `3`, identifiant `T1`, valeur `abc` puis `26.5`	La saisie `abc` est redemandée ; `T1 : 21.5 -> 26.5`
3	Choix `4` `P1`, puis `6` `pression`	`P1 desactive`, puis `Aucun capteur actif de type pression`
4	Capteur actif dont la valeur est **égale** à un seuil	Pas d'alarme

Livrables

- Le fichier `parc_capteurs.cpp`.
- Le tableau de conception (partie 1) et le compte rendu des tests.

Critères d'évaluation

Critère	Indicateurs de réussite	Points
Conception	Prototypes complets ; chaque passage de paramètre justifié correctement	5
Pointeurs	`chercherCapteur` renvoie une adresse ou `nullptr` ; tests systématiques ; usage de `->`	4
Fonctionnalités	Les six choix du menu conformes au cahier des charges	6
Saisies et robustesse	`lireReel` robuste ; identifiant inconnu traité dans les choix 2, 3 et 4	2
Qualité et tests	Aucune copie inutile, lisibilité, scénario et cas particuliers déroulés	3
Total		**20**